

Robotic Navigation HW3-1 Report

Deep Reinforcement Learning on Path Tracking

114062633 / Shin-Cheng Huang

April 11, 2026

1 Introduction

This report details the implementation of a Deep Reinforcement Learning (DRL) agent using Proximal Policy Optimization (PPO) for robotic path tracking. The task focuses on training a robust policy that enables a robot to follow a pre-defined path by processing 14-dimensional state observations and generating continuous steering commands.

2 Implementation of Required Tasks (TODO 1-5)

The implementation is divided into five key tasks as specified in the assignment requirements.

2.1 TODO 1: Policy Network Architecture

In `model.py`, the `PolicyNet` was implemented as a Multi-Layer Perceptrons (MLP). It consists of:

- Input Layer: Processes the 14-dimensional state space.
- Hidden Layers: Two fully connected layers with 64 units each, utilizing ReLU activation functions.
- Output Layer: Connected to a `DiagGaussian` module to output a probability distribution for the continuous action (steering).
- Initialization: Orthogonal initialization was applied to all weights with a gain factor calculated for ReLU to ensure stable gradient flow during early training.

2.2 TODO 2: Value Network Architecture

Also in `model.py`, the `ValueNet` (Critic) follows a similar MLP structure to the policy network:

- Shared Architecture: Two hidden layers of 64 units with ReLU activations.
- Output Layer: A single linear unit that predicts the state-value $V(s)$.
- Initialization: Consistent with the policy network, orthogonal initialization was used for weight consistency.

2.3 TODO 3: Multi-Environment Data Collection

The `EnvRunner.run` method in `env_runner.py` was implemented to collect experience batches. The logic includes:

- Converting state arrays to PyTorch tensors and moving them to the target `device` (CPU/GPU).
- Performing inference using `policy_net` and `value_net` within a `torch.no_grad()` block to save memory.
- Interacting with the environment using the `step` function across 8 parallel environments.
- Storing transitions (states, actions, log-probabilities, values, and rewards) into the mini-batch buffers.

2.4 TODO 4: PPO Training Logic and Loss Function

The core PPO update logic was implemented in `agent.py`:

- **Clipped Policy Loss:** Calculated the probability ratio $r_t(\theta)$ and implemented the surrogate objective $L^{CLIP} = \min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)$. This prevents the policy from updating too aggressively.
- **Value Loss:** Implemented a clipped Mean Squared Error (MSE) loss for the critic to ensure stable value estimation.
- **Optimization:** Used the Adam optimizer with gradient clipping to prevent exploding gradients.

2.5 TODO 5: Hyperparameter Configuration and Tuning

In `train.py`, hyperparameters were configured to balance training speed and convergence stability:

- **Parallelization:** `n_env = 8` to accelerate data collection.
- **Batch Size:** `n_step = 128` (resulting in a total batch of 1024 samples per update).
- **Optimization Parameters:** `lr = 1e-4`, `sample_n_epoch = 4`, and `sample_mb_size = 64`.
- **PPO Factors:** `clip_val = 0.2`, `gamma = 0.99`, and `lamb = 0.95` for GAE.

3 Experimental Results

The training results indicate successful convergence. The agent’s performance improved from an initial mean return of approximately 27 to a final return exceeding 200 after 30,000 iterations. This progression confirms that the implemented architecture and PPO logic are robust for the path tracking challenge.

4 Conclusion

By completing the five required tasks, a fully functional PPO-based robotic navigator was developed. The combination of an Actor-Critic architecture, GAE-based advantage estimation, and the clipped surrogate loss allowed the agent to learn precise path-following behaviors in a complex navigation environment.